WISKUNDE GEVORDERD · 🌍 BEYOND — 5DE EN 6DE MIDDELBAAR · UITLEG BIJ DE OEFENBUNDEL

# Programmeren: algoritmen en structuren

Bij elke oefening het antwoord en waarom het juist is. De nummers zijn dezelfde als in de oefenbundel.

---

- Dit blad is bedoeld om **samen na te lezen**, nadat je kind de bundel gemaakt heeft. Het hoort niet bij de bundel die je kind krijgt.
- Begin bij een fout antwoord met de vraag **hoe je kind eraan kwam**. Vaak zit de vergissing één stap eerder dan je denkt, en dan lees je hier samen die stap na.
- Staat er bij een oefening geen uitleg, dan zegt het antwoord zelf al genoeg.

## 1 De bouwstenen

Leer deze vier namen met een voorbeeld erbij. Een conditie kiest, een iteratie herhaalt; dat is het verschil dat het vaakst gevraagd wordt.

**1** Welke bouwsteen?

**a) een variabele · b) een constante · c) een conditie · d) een iteratie**

Let op het verschil tussen een variabele en een constante: niet de soort waarde maakt het verschil, maar of ze nog verandert.

**2** Hoe heet een stuk code dat je een naam geeft en hergebruikt?

**een functie**

Let erop dat een functie ook een eigen naam heeft en opnieuw opgeroepen kan worden. Dat is net wat haar nuttig maakt.

---

## 2 Het juiste sleutelwoord

Elk sleutelwoord hoort bij één bouwsteen. Koppel ze twee aan twee: if bij kiezen, for bij herhalen, def bij een functie, return bij haar antwoord.

**3** Welk sleutelwoord?

**a)** `if` **· b)** `for` **· c)** `def` **· d)** `return`

Let op de laatste twee: def maakt de functie, return geeft het resultaat terug. Een functie zonder return rekent iets uit en geeft niets.

## 3 De juiste datastructuur

Stel je twee vragen: telt de volgorde, en mag het veranderen? Daarmee kies je tussen een list, een tuple, een set en een dictionary.

**4** **een list; een dictionary; een set; een tuple**

Let op de laatste twee rijen: een set laat geen dubbels toe, en een tuple kan je niet meer wijzigen. Juist die eigenschap maakt ze hier de beste keuze.

## 4 Wat blijft er over in een set?

Een set houdt elke waarde maar één keer. Tel dus hoeveel verschillende waarden er in de lijst staan.

**5** Hoeveel?

**a)** $3$ · **b)** $4$ · **c)** $1$ · **d)** $3$

Let op de derde rij: vier keer hetzelfde getal geeft één element. En let erop dat de volgorde in een set geen rol speelt.

## 5 Waar of niet waar: datastructuren

Hou per structuur twee dingen in het oog: de volgorde en of ze aanpasbaar is. Daarmee beantwoord je bijna elke uitspraak.

**6** Een `tuple` kan je achteraf nog wijzigen.

**niet waar**

Niet waar: een tuple ligt vast. Wil je nog wijzigen, dan heb je een list nodig.

**7** Een `list` bewaart de volgorde van de elementen.

**waar**

**8** In een `set` kan hetzelfde element twee keer staan.

**niet waar**

Niet waar: in een set staat elk element precies één keer. Een tweede keer toevoegen verandert niets.

**9** Een string is een rij tekens.

**waar**

## 6 Algoritmische technieken

Leer deze vier technieken met hun kernwoord: zichzelf oproepen, opsplitsen, bewaren, halveren.

**10** Welke techniek?

**a) recursie · b) verdeel-en-heers · c) dynamisch programmeren · d) verdeel-en-heers**

Let erop dat halveren ook verdeel-en-heers is: je gooit elke stap de helft weg en lost verder in de andere helft op.

## 7 Recursie opschrijven

Een recursieve formule bestaat altijd uit twee delen: de stap die terugverwijst, en het stopgeval. Zonder het stopgeval stopt het programma nooit.

**11** De faculteit.

$n! = n \cdot (n-1)!$**, met** $0! = 1$ **als stopgeval.**

Let erop dat het stopgeval 0 faculteit gelijk aan 1 is, en niet 1 faculteit. Anders werkt de formule niet voor nul.

**12** De rij van Fibonacci.

$F(n) = F(n-1) + F(n-2)$**, met** $F(0) = 0$ **en** $F(1) = 1$**.**

Let erop dat Fibonacci twee stopgevallen nodig heeft, want de formule kijkt twee stappen terug.

## 8 Reken de recursie uit

Reken de rij van onder op in plaats van van boven af. Dan hou je de tussenresultaten bij en hoef je niets twee keer te doen.

**13** Hoeveel?

**a)** $24$ **· b)** $720$ **· c)** $5$ **· d)** $21$

Let erop dat de rij van Fibonacci veel trager groeit dan de faculteit: 6 faculteit is al 720, terwijl F van 8 maar 21 is.

## 9 Hoeveel stappen om te zoeken?

Elke stap halveert de rij. Tel dus hoeveel keer je door 2 kan delen tot er één getal overblijft; dat is de logaritme met grondtal 2.

**14** Hoeveel stappen?

**a)** $3$ · **b)** $4$ · **c)** $6$ · **d)** $10$

Let erop dat 1024 getallen maar 10 stappen vragen. Dat is precies waarom halveren zo veel sneller is dan één voor één doorlopen.

---

## 10 Dynamisch programmeren

Schrijf de boom van oproepen eerst uit. Dan zie je zelf welke waarden meer dan één keer berekend worden.

**15** Welke oproepen heeft een programma nodig om $F(4)$ recursief uit te rekenen, en welke waarden berekent het meer dan één keer?

**$F(4)$ roept $F(3)$ en $F(2)$ op, $F(3)$ roept $F(2)$ en $F(1)$ op, en $F(2)$ roept telkens $F(1)$ en $F(0)$ op. Zo bereken je $F(2)$ twee keer en $F(1)$ drie keer.**

Let erop dat je de oproepen echt uitschrijft: zo zie je dat F van 1 drie keer berekend wordt, en dat is precies de verspilling.

**16** Dynamisch programmeren kost meer geheugen. Wat levert het op, en wanneer is dat de moeite?

**Het wint tijd, want je berekent niets twee keer. Bij een probleem waar dezelfde deelberekening vaak terugkomt, zoals de rij van Fibonacci, scheelt dat enorm.**

## 11 Correctheid en eindigheid

Hou de twee woorden uit elkaar: correct gaat over het antwoord, eindig over het stoppen. Een programma heeft ze allebei nodig.

**17** Leg het verschil uit tussen de correctheid en de eindigheid van een algoritme.

**Correct betekent dat het voor elke toegelaten invoer het juiste antwoord geeft. Eindig betekent dat het na een eindig aantal stappen stopt. Een programma kan correct geredeneerd zijn en toch eeuwig blijven lopen.**

Let erop dat de twee onafhankelijk zijn: een algoritme kan juist redeneren en toch eeuwig blijven lopen.

**18** Hoe beargumenteer je de eindigheid van een lus, en hoe die van een recursieve functie?

**Bij een lus toon je dat de voorwaarde ooit vals wordt. Bij recursie toon je dat je het stopgeval altijd bereikt.**

---

## 12 Waar of niet waar: algoritmen

Een algoritme is een werkwijze, niet een taal. Dat idee zit achter bijna elke uitspraak in deze reeks.

**19** Een algoritme is pas een algoritme als het in Python geschreven staat.

**niet waar**

Niet waar: een algoritme is een werkwijze in eindig veel stappen. Je kan ze ook in woorden of in een schema zetten.

**20** Twee algoritmen die hetzelfde antwoord geven, zijn ook even snel.

**niet waar**

Niet waar: hetzelfde antwoord kan je op een snelle en op een trage manier vinden. Kijk naar de laatste doordenker van deze bundel.

**21** Een algoritme is eindig als het na een eindig aantal stappen stopt.

**waar**

**22** Je test je programma ook met randgevallen, omdat fouten zich daar verstoppen.

**waar**

## 13 Bibliotheken en bestanden

Elke bibliotheek doet één soort werk. Koppel de naam aan die taak en je hoeft niets van buiten te leren.

**23** **grafieken tekenen; vlot met grote rijen getallen rekenen; toevalsgetallen genereren**

**24** Welk soort bestand lees je naast een gewoon tekstbestand in en schrijf je weg?

**een CSV-bestand**

Let erop dat een CSV-bestand gewoon tekst is met komma's ertussen. Daarom kan je het met een rekenblad openen en met een programma inlezen.

---

## 14 Groeigedrag

Vul n in de formule in en reken. Bij een logaritme met grondtal 2 is dat ongeveer 10 bij duizend.

**25** **ongeveer** $10$**; ongeveer** $1000$**; ongeveer** $1\,000\,000$

Let erop dat het verschil enorm is: 10 tegenover een miljoen bij hetzelfde aantal gegevens. Daarom kijk je naar het groeigedrag en niet naar de klok.

---

## 15 De Riemannsom

Een Riemannsom telt stroken op: hoogte maal breedte, en dat n keer. Schrijf eerst op wat delta x is.

**26** Schrijf de benadering van $\int_a^b f(x)\,dx$ op als een som, en zeg wat $\Delta x$ is.

$\int_a^b f(x)\,dx \approx \sum_{i=1}^{n} f(x_i)\cdot \Delta x$**, met** $\Delta x = \dfrac{b-a}{n}$**, de breedte van één strook.**

**27** Je verdeelt $[0, 4]$ in $8$ stroken. Hoe breed is elke strook, en wat gebeurt er met de benadering als je $n$ verdubbelt?

$\Delta x = \dfrac{4-0}{8} = 0{,}5$**. Verdubbel je** $n$**, dan worden de stroken half zo smal en wordt de benadering beter.**

Let erop dat smallere stroken de benadering beter maken, niet slechter. Dat is het idee achter de integraal: de limiet voor n naar oneindig.

## 16 Uitleggen

Noem bij een uitlegvraag ook waarom het telt voor wie je code leest. Leesbaarheid is een deel van de beoordeling en geen opsmuk.

**28** Waarom geef je variabelen een zinvolle naam, en waarvoor dient commentaar in je code?

**Allebei om je code leesbaar te houden voor jezelf en voor anderen, en dat telt mee in de beoordeling. Commentaar wordt niet mee uitgevoerd; ze staat er alleen voor wie de code leest.**

**29** Noem de vier stappen waarin je een oplossing uitwerkt, en leg uit waarom je niet meteen begint te typen.

**Analyseren, een algoritme ontwerpen, programmeren, en testen en debuggen. Meteen beginnen typen zonder het probleem te analyseren kost je achteraf meer tijd dan het wint.**

Let erop dat testen en debuggen er echt bij horen: zonder die vierde stap weet je niet of je programma werkt.

## 17 Doordenkers

Denk bij deze twee aan de voorwaarde en aan het groeigedrag. Een techniek werkt alleen als haar voorwaarde klopt.

**30** Een lijst van een miljoen namen is niet gesorteerd en je moet er één naam in terugvinden. Mag je hier halveren?

**Nee: halveren werkt alleen op een gesorteerde rij. Je loopt de lijst dan van voor naar achter door, of je sorteert ze eerst als je nog vaak moet zoeken.**

Let erop dat halveren een gesorteerde rij veronderstelt. Sorteren kost zelf tijd, dus dat is pas de moeite als je vaak moet zoeken.

**31** Twee programma's geven hetzelfde antwoord. Het ene doet $O(n)$ stappen, het andere $O(n^2)$. Bij $n = 10$ merk je nauwelijks verschil. Hoe komt dat, en wanneer wel?

**Bij $n = 10$ is dat $10$ tegenover $100$ stappen, en dat duurt allebei niets. Bij $n = 1000$ wordt het $1000$ tegenover $1\,000\,000$, en dan wordt het tweede onbruikbaar.**

Let erop dat je de twee bij een klein en bij een groot aantal vergelijkt. Bij kleine aantallen zie je het verschil niet, en dat maakt een trage aanpak verraderlijk.